claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_2be29ecf-a31\agent-ae113fa42ad2b5ef6.jsonl`
- SHA-256: `5ccbe449db6470e028c522eceaa97ae1682cc079760d47f821cbd0200c2a1375`
- Source modified: `2026-06-22T01:29:08+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_2be29ecf-a31`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T01:28:08.300Z)

```
Adversarially verify this P2b-install review finding. Read the cited code in
C:\\Users\\avidu\\Projects\\bhi-p2b yourself (run `cd C:\\Users\\avidu\\Projects\\bhi-p2b && git
diff origin/master...HEAD` and open the file). Decide if it is a REAL defect that should block
or be fixed, or a false alarm / nit. Default to is_real=false if it does not hold up.

Lens: safety-dataloss
Severity(claimed): major
Title: uninstall.py deletes manifest 'created'/'user_data' paths verbatim with NO containment
check ? a corrupted/edited/foreign manifest can rmtree an arbitrary directory (incl. on a
default, non-purge run)
File: scripts/uninstall.py
Detail: The manifest is plain JSON read from disk (main(), line 94) and is fully user-editable /
corruptible / swappable via --manifest. Its 'created' list is deleted on EVERY uninstall (lines
119-122, not just --purge), and 'user_data' on --purge (lines 124-127), by passing each path
straight to _rm() -> shutil.rmtree (lines 68-79). Nothing verifies these paths live under
app_home. If 'created' (or 'user_data') ever contains a broad path ? e.g. a hand-edit, a future
installer bug, a stale manifest from a different/old app-home, or app_home itself ? a plain
`python uninstall.py` (no --purge) will rmtree it. _rm only guards against symlinked dirs, not
against a too-broad/outside-app-home real directory. This is the classic 'trust the deletion
manifest blindly' data-loss vector and should not depend on the manifest being pristine. The
deletion orchestration (main/_rm) is also entirely untested ? tests/test_p2b_install.py only
exercises pure helpers (build_manifest/shortcut_spec/argv), never a real or adversarial
deletion.
Suggested fix: Before deleting, resolve app_home and refuse any path that is not strictly
contained within it. Add a guard in _rm (or at the call sites) like: `rp = path.resolve(); ah =
app_home.resolve(); if rp == ah or not rp.is_relative_to(ah): print('refusing to remove path
outside app-home'); return` ? and additionally refuse to rmtree app_home itself, the filesystem
root, or the user home. Apply to BOTH the 'created' loop and the --purge 'user_data' loop. Add a
test that feeds a manifest whose 'created'/'user_data' contains an outside-app-home dir and
asserts it is NOT removed.
```

2. assistant (2026-06-22T01:28:11.097Z)

```
I'll verify this finding by examining the actual code in the cited repository.
```

3. user (2026-06-22T01:28:14.706Z)

```
78e423d docs(pkg): link P2b-install PR #310 in the §7 tracker
a5c1493 feat(pkg): P2b-install ? batteries-included install + truly-local 'motion' first-run
default (gate O8)
```

4. user (2026-06-22T01:28:14.892Z)

```
 README.md                     |  31 +++++++
```

```
 backend/config/__init__.py |   9 +-
 backend/config/loader.py   |  34 ++++++++
 backend/main.py            |  15 ++++
 docs/PACKAGING_PLAN.md     |  13 +--
 scripts/install.py         | 210 ++++++++++++++++++++++++++++++++++++++++++++
 scripts/uninstall.py       | 150 ++++++++++++++++++++++++++++++++
 tests/test_app_launcher.py |  42 +++++++++
 tests/test_p2b_install.py  | 163 +++++++++++++++++++++++++++++++++++
 9 files changed, 660 insertions(+), 7 deletions(-)
```

5. assistant (2026-06-22T01:28:17.107Z)

Now let me read the full uninstall.py file.

6. user (2026-06-22T01:28:17.997Z)

```
#!/usr/bin/env python3
"""Clean uninstaller for the KhelSutra highlight indexer (PACKAGING_PLAN.md P2b).

Reverses ``scripts/install.py`` from the manifest it wrote:

  * uninstalls the package (``uv tool uninstall`` / ``pip uninstall -y`` ? matches the install method);
  * removes the files the installer created (launcher shortcut, the copied uninstaller, the manifest);
  * by default **KEEPS your data** (the seeded ``config.json``, your ``.env`` with the Gemini key,
    and the ``output/`` reels + DB). Pass ``--purge`` to delete those too.

Usage::

    python uninstall.py                 # find the manifest in the app-home, keep user data
    python uninstall.py --purge         # also delete config.json / .env / output (your data)
    python uninstall.py --manifest /path/to/uninstall_manifest.json
    python uninstall.py --dry-run       # print the plan, change nothing

stdlib-only; safe to run from the copy the installer dropped in the app-home.
"""
from __future__ import annotations

import argparse
import json
import os
import platform
import shutil
import subprocess
import sys
from pathlib import Path

APP_DIR_NAME = "Khelsutra"
MANIFEST_NAME = "uninstall_manifest.json"


def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
                        env: dict[str, str] | None = None) -> Path:
    """Mirror of the installer's app-home resolver (used to LOCATE the manifest when not given)."""
    system = system or platform.system()
    e = os.environ if env is None else env
    if system == "Windows":
        base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
    elif system == "Darwin":
        base = str(Path.home() / "Library" / "Application Support")
```

```
    else:
        base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
    return Path(base) / app_name


def find_manifest(explicit: str | None) -> Path | None:
    """The manifest path: explicit arg, else next to THIS file, else the default app-home."""
    if explicit:
        p = Path(explicit)
        return p if p.exists() else None
    for cand in (Path(__file__).resolve().parent / MANIFEST_NAME,
                 default_os_app_home() / MANIFEST_NAME):
        if cand.exists():
            return cand
    return None


def uninstall_command(method: str, package: str) -> list[str]:
    """The argv to remove the package (PURE)."""
    if method == "uv":
        return ["uv", "tool", "uninstall", package]
    return [sys.executable, "-m", "pip", "uninstall", "-y", package]


def _rm(path: Path, dry: bool) -> None:
    try:
        if path.is_dir() and not path.is_symlink():
            print(f"  - rmtree {path}")
            if not dry:
                shutil.rmtree(path, ignore_errors=True)
        elif path.exists() or path.is_symlink():
            print(f"  - remove {path}")
            if not dry:
                path.unlink(missing_ok=True)
    except OSError as e:
        print(f"  ! could not remove {path}: {e}")


def main(argv: list[str] | None = None) -> int:
    ap = argparse.ArgumentParser(description="Uninstall the KhelSutra highlight indexer.")
    ap.add_argument("--manifest", default=None, help="Path to uninstall_manifest.json.")
    ap.add_argument("--purge", action="store_true",
                    help="Also delete YOUR data (config.json, .env, output/). Default keeps it.")
    ap.add_argument("--dry-run", action="store_true", help="Print the plan; change nothing.")
    args = ap.parse_args(argv)

    manifest_path = find_manifest(args.manifest)
    if not manifest_path:
        print("[ERROR] no uninstall_manifest.json found (pass --manifest <path>).")
        return 1
    manifest = json.loads(manifest_path.read_text(encoding="utf-8"))
    method = manifest.get("install_method", "pip")
    package = manifest.get("package", "badminton-highlight-indexer")
    created = [Path(p) for p in manifest.get("created", [])]
    user_data = [Path(p) for p in manifest.get("user_data", [])]
    app_home = Path(manifest.get("app_home", str(manifest_path.parent)))

    print(f"KhelSutra uninstaller ? {package} (installed via {method})")
    print(f"  manifest : {manifest_path}")
    print(f"  app-home : {app_home}")
    print(f"  data     : {'WILL BE DELETED (--purge)' if args.purge else 'kept (use
```

```
--purge to delete)'}")
    dry = args.dry_run

    print("\n[1/3] removing the package?")
    cmd = uninstall_command(method, package)
    print(f"  $ {' '.join(cmd)}")
    if not dry:
        try:
            subprocess.run(cmd, check=False)
        except FileNotFoundError as e:
            print(f"  ! {method} not found ({e}); remove the package manually.")

    print("\n[2/3] removing installer-created files?")
    # Remove the manifest LAST (we still reference it); skip the running uninstaller copy itself.
    self_path = Path(__file__).resolve()
    for p in created:
        if p.resolve() == self_path or p.resolve() == manifest_path.resolve():
            continue
        _rm(p, dry)

    if args.purge:
        print("\n[3/3] purging user data (--purge)?")
        for p in user_data:
            _rm(p, dry)
    else:
        print("\n[3/3] keeping your data:")
        for p in user_data:
            print(f"  · {p}")

    # Manifest + (optionally) the empty app-home last.
    _rm(manifest_path, dry)
    if args.purge and app_home.exists():
        try:
            remaining = [c for c in app_home.iterdir() if c.resolve() != self_path]
            if not remaining:
                _rm(app_home, dry)
        except OSError:
            pass

    print("\n? Uninstalled." + ("" if args.purge else " Your data was kept."))
    if self_path.parent == app_home:
        print(f"   (you can delete this script + the app-home folder manually: {app_home})")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())

```

7. assistant (2026-06-22T01:28:20.862Z)

Now let me look at how install.py builds the manifest to understand what `created` and `user_data` actually contain.

8. user (2026-06-22T01:28:21.737Z)

```
#!/usr/bin/env python3
"""Batteries-included installer for the KhelSutra highlight indexer (PACKAGING_PLAN.md P2b).

Cross-platform (Windows / macOS / Linux), no native component (O1):

  * installs the LIGHT (torch-free) package via ``uv tool install`` (preferred ? an
```

```
isolated tool
    env that puts ``indexer`` on PATH) or ``pip install`` (fallback);
  * creates a per-OS launcher shortcut that pins ``RALLY_APP_HOME`` to the OS app-data dir and runs
    ``indexer --app`` ? so config / .env / DB / reels live in ONE stable place
    (mirrors ``backend.utils.app_home.default_os_app_home``). Pinning the env BEFORE the process
    starts is load-bearing: the engine's ``load_dotenv`` runs at import, so only an env set by the
    launcher (not by the engine) roots the ``.env`` in the app-home;
  * writes an uninstall manifest so ``scripts/uninstall.py`` can cleanly reverse it.

The truly-local + ``motion`` default config is seeded by the ENGINE itself on the first
``indexer --app`` run (``write_light_default_config``), so it works regardless of install method;
this script only wires the install + the launcher.

Usage::

    python scripts/install.py                       # install '.' (this checkout), uv-if-present else pip
    python scripts/install.py --target badminton-highlight-indexer   # from PyPI (once published)
    python scripts/install.py --method pip          # force pip
    python scripts/install.py --dry-run             # print the plan, change nothing

stdlib-only so it runs as a standalone bootstrap BEFORE the package exists.
"""
from __future__ import annotations

import argparse
import json
import os
import platform
import shutil
import subprocess
import sys
from datetime import datetime, timezone
from pathlib import Path

PACKAGE = "badminton-highlight-indexer"
APP_DIR_NAME = "Khelsutra"  # mirrors backend.utils.app_home.APP_DIR_NAME
MANIFEST_NAME = "uninstall_manifest.json"


# --- pure helpers (unit-tested) ------------------------------------------------------------

def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
                        env: dict[str, str] | None = None) -> Path:
    """The OS-conventional per-user app-data dir ? a standalone mirror of
    ``backend.utils.app_home.default_os_app_home`` (this script runs before the package exists)."""
    system = system or platform.system()
    e = os.environ if env is None else env
    if system == "Windows":
        base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
    elif system == "Darwin":
        base = str(Path.home() / "Library" / "Application Support")
    else:
        base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
    return Path(base) / app_name


def shortcut_spec(system: str, app_home: Path, indexer_cmd: str = "indexer") ->
```

```
tuple[str, str, bool]:
    """Return ``(filename, content, make_executable)`` for the per-OS launcher shortcut.

    The shortcut pins ``RALLY_APP_HOME`` then runs ``indexer --app``. PURE (no IO) so it's testable.
    """
    home = str(app_home)
    if system == "Windows":
        content = (
            "@echo off\r\n"
            "rem KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the app.\r\n"
            f'set "RALLY_APP_HOME={home}"\r\n'
            f"{indexer_cmd} --app\r\n"
        )
        return "KhelSutra.bat", content, False
    if system == "Darwin":
        content = (
            "#!/bin/bash\n"
            "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the app.\n"
            f'export RALLY_APP_HOME="{home}"\n'
            f"exec {indexer_cmd} --app\n"
        )
        return "KhelSutra.command", content, True
    # Linux / other
    content = (
        "#!/bin/bash\n"
        "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the app.\n"
        f'export RALLY_APP_HOME="{home}"\n'
        f"exec {indexer_cmd} --app\n"
    )
    return "khelsutra.sh", content, True


def build_manifest(*, method: str, app_home: Path, shortcut: Path, created: list[Path],
                   timestamp: str) -> dict:
    """The uninstall manifest (PURE). ``created`` = files THIS installer made (safe to remove);
    ``user_data`` = paths the engine/user own (kept unless ``uninstall.py --purge``)."""
    return {
        "schema": 1,
        "product": "KhelSutra highlight indexer",
        "package": PACKAGE,
        "install_method": method,            # "uv" | "pip"
        "indexer_command": "indexer",
        "app_home": str(app_home),
        "shortcut": str(shortcut),
        "created": [str(p) for p in created],
        "user_data": [
            str(app_home / "config.json"),
            str(app_home / ".env"),
            str(app_home / "output"),
        ],
        "timestamp": timestamp,
    }


def resolve_method(requested: str, uv_present: bool) -> str:
    """Pick the installer: explicit beats auto; auto prefers uv when present, else pip."""
    if requested in ("uv", "pip"):
        return requested
    return "uv" if uv_present else "pip"
```

```python


def install_command(method: str, target: str) -> list[str]:
    """The argv for the chosen installer (PURE)."""
    if method == "uv":
        return ["uv", "tool", "install", target]
    return [sys.executable, "-m", "pip", "install", target]


# --- IO / orchestration --------------------------------------------------------------------

def _run(argv: list[str]) -> None:
    print(f"  $ {' '.join(argv)}")
    subprocess.run(argv, check=True)


def main(argv: list[str] | None = None) -> int:
    ap = argparse.ArgumentParser(description="Install the KhelSutra highlight indexer (LIGHT tier).")
    ap.add_argument("--target", default=".",
                    help="What to install: '.' (this checkout, default) or a PyPI name/spec.")
    ap.add_argument("--method", choices=("auto", "uv", "pip"), default="auto",
                    help="Installer to use (default: uv if present, else pip).")
    ap.add_argument("--dry-run", action="store_true", help="Print the plan; change nothing.")
    args = ap.parse_args(argv)

    system = platform.system()
    app_home = default_os_app_home()
    method = resolve_method(args.method, shutil.which("uv") is not None)
    cmd = install_command(method, args.target)
    fname, content, executable = shortcut_spec(system, app_home)
    shortcut = app_home / fname

    print(f"KhelSutra installer ? {PACKAGE} (LIGHT, torch-free)")
    print(f"  OS         : {system}")
    print(f"  installer  : {method}")
    print(f"  target     : {args.target}")
    print(f"  app-home   : {app_home}")
    print(f"  shortcut   : {shortcut}")
    if args.dry_run:
        print("\n[dry-run] would run:")
        print(f"  $ {' '.join(cmd)}")
        print(f"  write shortcut -> {shortcut}")
        print(f"  write manifest -> {app_home / MANIFEST_NAME}")
        print(f"  copy uninstaller -> {app_home / 'uninstall.py'}")
        return 0

    print("\n[1/3] installing the package?")
    try:
        _run(cmd)
    except (subprocess.CalledProcessError, FileNotFoundError) as e:
        print(f"[ERROR] install failed: {e}")
        if method == "uv":
            print("  (uv not usable? re-run with --method pip)")
        return 1

    print("\n[2/3] writing the launcher shortcut + app-home?")
    app_home.mkdir(parents=True, exist_ok=True)
    shortcut.write_text(content, encoding="utf-8")
    if executable:
        shortcut.chmod(0o755)
    # Make the uninstaller available next to the data it removes.
```

```
    created: list[Path] = [shortcut]
    here = Path(__file__).resolve().parent
    src_uninstall = here / "uninstall.py"
    dst_uninstall = app_home / "uninstall.py"
    if src_uninstall.exists():
        shutil.copyfile(src_uninstall, dst_uninstall)
        created.append(dst_uninstall)

    print("\n[3/3] writing the uninstall manifest?")
    manifest_path = app_home / MANIFEST_NAME
    created.append(manifest_path)
    manifest = build_manifest(
        method=method, app_home=app_home, shortcut=shortcut, created=created,
        timestamp=datetime.now(timezone.utc).isoformat(timespec="seconds"),
    )
    manifest_path.write_text(json.dumps(manifest, indent=2) + "\n", encoding="utf-8")

    print("\n? Installed. To launch KhelSutra:")
    if system == "Windows":
        print(f"   double-click  {shortcut}   (or run `indexer --app`)")
    else:
        print(f"   {shortcut}   (or run `indexer --app`)")
    print("   The first run seeds a truly-local 'motion' config (offline, no key/GPU needed).")
    print(f"   To uninstall:  python \"{dst_uninstall}\"   (add --purge to also delete your data)")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

9. assistant (2026-06-22T01:28:24.038Z)

Now let me check the test file to verify the claim that deletion orchestration is untested.

10. user (2026-06-22T01:28:24.841Z)

```
"""P2b-install (PACKAGING_PLAN.md): batteries-included config seeding + install/uninstall tooling.

Covers:
  * the engine-side ``write_light_default_config`` (gate O8 default = truly-local + motion), incl.
    the load-bearing **no-overwrite** idempotency and the EFFECTIVE config the loader produces;
  * the pure logic of ``scripts/install.py`` / ``scripts/uninstall.py`` (shortcut content, manifest,
    installer/uninstaller argv, method resolution, OS app-home) ? no real subprocess/IO.
"""
from __future__ import annotations

import importlib.util
import json
from pathlib import Path

import pytest

from backend.config import LIGHT_DEFAULT_CONFIG, load_config, write_light_default_config


def _load_script(name: str):
    """Load a scripts/*.py bootstrap file as a module (it is stdlib-only, safe to exec)."""
```

```python
        path = Path(__file__).resolve().parents[1] / "scripts" / name
        spec = importlib.util.spec_from_file_location(name[:-3], path)
        assert spec and spec.loader
        mod = importlib.util.module_from_spec(spec)
        spec.loader.exec_module(mod)
        return mod


    # --- engine seeding: write_light_default_config (gate O8) ---------------------------------

    def test_seed_writes_truly_local_motion(tmp_path):
        cfg = tmp_path / "sub" / "config.json"   # also proves parent-dir creation
        path, written = write_light_default_config(cfg)
        assert written is True and path == cfg and cfg.exists()
        data = json.loads(cfg.read_text(encoding="utf-8"))
        assert data == LIGHT_DEFAULT_CONFIG
        assert data["indexing"]["default_segmenter"] == "motion"
        assert data["indexing"]["use_gpu"] is False
        assert data["deployment"]["profile"] == "truly-local"


    def test_seed_does_not_overwrite_existing(tmp_path):
        cfg = tmp_path / "config.json"
        cfg.write_text('{"sport": "tennis", "indexing": {"default_segmenter": "gemini"}}', encoding="utf-8")
        path, written = write_light_default_config(cfg)
        assert written is False and path == cfg
        # The user's file is preserved untouched (the wizard/edits are never clobbered).
        assert json.loads(cfg.read_text(encoding="utf-8"))["indexing"]["default_segmenter"] == "gemini"


    def test_seed_overwrite_true_forces(tmp_path):
        cfg = tmp_path / "config.json"
        cfg.write_text("{}", encoding="utf-8")
        _, written = write_light_default_config(cfg, overwrite=True)
        assert written is True
        assert json.loads(cfg.read_text(encoding="utf-8")) == LIGHT_DEFAULT_CONFIG


    def test_seeded_config_loads_to_truly_local_motion(tmp_path):
        """The minimal seeded file, run through the loader, expands via the truly-local PRESET to a
        coherent torch-free/offline config (motion + no Gemini handoff + local storage + no GPU)."""
        cfg = tmp_path / "config.json"
        write_light_default_config(cfg)
        loaded = load_config(cfg)
        assert loaded.indexing.default_segmenter == "motion"
        assert loaded.indexing.use_gpu is False
        assert loaded.deployment.profile == "truly-local"
        assert loaded.indexing.skip_ai_handoff is True       # supplied by the truly-local preset
        assert loaded.storage.backend == "local"             # supplied by the truly-local preset


    # --- scripts/install.py pure logic ---------------------------------------------------------

    @pytest.fixture(scope="module")
    def install_mod():
        return _load_script("install.py")

```

```python

@pytest.fixture(scope="module")
def uninstall_mod():
    return _load_script("uninstall.py")


@pytest.mark.parametrize(
    "system, fname, executable",
    [("Windows", "KhelSutra.bat", False), ("Darwin", "KhelSutra.command", True),
     ("Linux", "khelsutra.sh", True)],
)
def test_shortcut_spec_per_os(install_mod, system, fname, executable):
    home = Path("/home/u/AppHome")
    name, content, ex = install_mod.shortcut_spec(system, home)
    assert name == fname and ex is executable
    # str(home) is OS-native (backslashes on Windows) ? assert against it, not a hardcoded literal.
    assert "RALLY_APP_HOME" in content and str(home) in content
    assert "indexer --app" in content


def test_resolve_method(install_mod):
    assert install_mod.resolve_method("auto", uv_present=True) == "uv"
    assert install_mod.resolve_method("auto", uv_present=False) == "pip"
    assert install_mod.resolve_method("pip", uv_present=True) == "pip"   # explicit wins
    assert install_mod.resolve_method("uv", uv_present=False) == "uv"


def test_install_command_argv(install_mod):
    assert install_mod.install_command("uv", ".") == ["uv", "tool", "install", "."]
    pip = install_mod.install_command("pip", "badminton-highlight-indexer")
    assert pip[1:] == ["-m", "pip", "install", "badminton-highlight-indexer"]


def test_default_os_app_home_per_os(install_mod):
    win = install_mod.default_os_app_home(system="Windows", env={"APPDATA": r"C:\Users\u\AppData\Roaming"})
    assert win.name == "Khelsutra" and "Roaming" in str(win)
    lin = install_mod.default_os_app_home(system="Linux", env={"XDG_DATA_HOME": "/home/u/.local/share"})
    assert str(lin) == str(Path("/home/u/.local/share") / "Khelsutra")


def test_build_manifest_shape(install_mod):
    home = Path("/home/u/AppHome")
    m = install_mod.build_manifest(
        method="uv", app_home=home, shortcut=home / "khelsutra.sh",
        created=[home / "khelsutra.sh", home / "uninstall_manifest.json"], timestamp="2026-06-22T00:00:00+00:00",
    )
    assert m["package"] == "badminton-highlight-indexer"
    assert m["install_method"] == "uv"
    assert str(home / "config.json") in m["user_data"]   # user data is tracked separately (kept on uninstall)
    assert m["shortcut"] == str(home / "khelsutra.sh")
    assert m["schema"] == 1


# --- scripts/uninstall.py pure logic -------------------------------------------------------

def test_uninstall_command_argv(uninstall_mod):
    assert uninstall_mod.uninstall_command("uv", "badminton-highlight-indexer") == \
        ["uv", "tool", "uninstall", "badminton-highlight-indexer"]
    pip = uninstall_mod.uninstall_command("pip", "badminton-highlight-indexer")
```

```
        assert pip[1:] == ["-m", "pip", "uninstall", "-y", "badminton-highlight-indexer"]


    def test_find_manifest_explicit(uninstall_mod, tmp_path):
        m = tmp_path / "uninstall_manifest.json"
        m.write_text("{}", encoding="utf-8")
        assert uninstall_mod.find_manifest(str(m)) == m
        assert uninstall_mod.find_manifest(str(tmp_path / "nope.json")) is None


    def test_install_uninstall_roundtrip_manifest(install_mod, uninstall_mod, tmp_path):
        """The manifest install.py would write is consumable by uninstall.py's expectations."""
        home = tmp_path / "AppHome"
        m = install_mod.build_manifest(
            method="pip", app_home=home, shortcut=home / "KhelSutra.bat",
            created=[home / "KhelSutra.bat", home / "uninstall_manifest.json"],
            timestamp="2026-06-22T00:00:00+00:00",
        )
        mp = home / "uninstall_manifest.json"
        home.mkdir(parents=True)
        mp.write_text(json.dumps(m), encoding="utf-8")
        assert uninstall_mod.find_manifest(str(mp)) == mp
        loaded = json.loads(mp.read_text(encoding="utf-8"))
        assert uninstall_mod.uninstall_command(loaded["install_method"], loaded["package"])[0:3] in (
            ["uv", "tool", "uninstall"], [install_mod.sys.executable, "-m", "pip"],
        )
```

11. assistant (2026-06-22T01:28:42.509Z)

I have now read all the relevant code. Let me analyze the finding carefully.

The finding's factual claims are accurate:
- `manifest` is plain JSON read from disk (line 94) ? true.
- `--manifest` is fully user-supplyable (line 84) ? true.
- `created` paths deleted on EVERY uninstall (lines 119-122) ? true.
- `user_data` deleted only on `--purge` (lines 124-127) ? true.
- `_rm` -> `shutil.rmtree` (lines 68-73), only guards against symlinked dirs ? true.
- No containment check against `app_home` ? true. `app_home` is read but only used for display (lines 99, 104) and the final empty-dir cleanup (line 135).
- Tests only exercise pure helpers, never `main`/`_rm` deletion ? true.

Now the adversarial lens ? is this a REAL defect that should block/be fixed, or a false alarm / nit?

Let me weigh the realism of the threat model:

1. **Who writes the manifest?** The installer (`install.py`) writes it with `created` = `[shortcut, dst_uninstall, manifest_path]` ? all narrowly scoped under `app_home`, and `user_data` = three specific paths under `app_home`. So in the normal install?uninstall flow, every path is already contained. The danger only materializes with a corrupted/hand-edited/foreign/swapped manifest.

2. **Is this a "trust the deletion manifest blindly" data-loss vector?** Yes, structurally. A defense-in-depth containment check is a well-established, cheap, standard hardening for uninstallers that rmtree from a manifest. This matches a real security/safety pattern.

3. **Severity calibration.** The claimed severity is "major." Let me consider counterarguments to that:
   - The trigger requires a non-pristine manifest (corruption, manual edit, a future installer bug, or pointing `--manifest` at a stale/foreign manifest). This is not the normal path ? it's a "if something else goes wrong, this amplifies it" hardening gap.
   - However, the repo's own conventions explicitly elevate exactly this class: "loud failures,

no silent drops," "data-loss" sensitivity, and the launch-quality bar. An uninstaller that can `rmtree` an arbitrary directory (including `app_home` itself, or a broad path) on a plain non-purge run is a genuine data-loss footgun. The "default (non-purge) run can still delete" angle is correct ? `created` is deleted unconditionally.

Let me double check the one nuance the finding raises: can `created` realistically contain `app_home` itself or a broad path? In the current installer, no. But the finding's point is about robustness against a non-pristine manifest, which is the legitimate concern for a destructive tool. The `_rm` only skips symlinked dirs, and `ignore_errors=True` on rmtree means it won't even error out partway ? it'll silently delete what it can.

Let me verify there's genuinely no containment guard anywhere (e.g., in a helper I might have missed) and confirm `is_relative_to` availability given the Python 3.8+ requirement.

12. user (2026-06-22T01:28:44.517Z)

```
35:def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
54:    for cand in (Path(__file__).resolve().parent / MANIFEST_NAME,
55:                 default_os_app_home() / MANIFEST_NAME):
99:    app_home = Path(manifest.get("app_home", str(manifest_path.parent)))
103:    print(f"  app-home : {app_home}")
118:    self_path = Path(__file__).resolve()
120:        if p.resolve() == self_path or p.resolve() == manifest_path.resolve():
135:    if args.purge and app_home.exists():
137:            remaining = [c for c in app_home.iterdir() if c.resolve() != self_path]
139:                _rm(app_home, dry)
144:    if self_path.parent == app_home:
145:        print(f"   (you can delete this script + the app-home folder manually: {app_home})")
```

13. user (2026-06-22T01:28:45.643Z)

pyproject.toml:21:requires-python = ">=3.10"

14. assistant (2026-06-22T01:28:51.234Z)

This confirms the analysis. Two important findings:

1. **No containment guard exists** ? `app_home` is read but used only for display (lines 103, 145) and the trailing empty-dir cleanup (lines 135-139). The deletion loops (`created`, `user_data`) pass paths straight to `_rm` with no relative-to-`app_home` check.

2. **The suggested fix is implementable** ? `requires-python = ">=3.10"` for the package. `Path.is_relative_to` was added in Python 3.9, so the suggested guard works. (Minor caveat: the install/uninstall scripts are "stdlib-only bootstrap" scripts that run before the package, but they target the same Python; 3.10+ is a safe assumption, and a fallback is trivial.)

I have enough to render a verdict.

15. user (2026-06-22T01:29:02.322Z)

Structured output provided successfully

16. assistant (2026-06-22T01:29:08.878Z)

Verdict: **REAL but minor** (downgraded from the claimed major).

The finding is factually accurate on every point I checked in
`C:\Users\avidu\Projects\bhi-p2b\scripts\uninstall.py`:
- The manifest is untrusted input (plain JSON, `--manifest`-supplyable, line 94 / line 84).
- `created` is deleted unconditionally on every run (lines 119-122); `user_data` on `--purge` (lines 124-127); both via `_rm` ? `shutil.rmtree` with `ignore_errors=True` (lines 68-73).
- There is genuinely no containment check ? `app_home` (line 99) is used only for display and the trailing empty-dir cleanup, never as a deletion boundary.
- The deletion orchestration is untested; `tests/test_p2b_install.py` only covers pure helpers.

The suggested `is_relative_to(app_home)` guard is feasible (`requires-python >=3.10`).

I downgraded the severity because the only writer of these manifests, `install.py::build_manifest` (lines 94-113), scopes both lists to narrow, specific files under `app_home`, so the normal install?uninstall flow is already safe. Data loss requires a non-pristine/foreign manifest containing a broad path ? a real "trust the deletion manifest blindly" hardening gap worth fixing in this PR, but not a defect that fires under documented usage, so it should not block as a major.